COLLEGE ENGINEERING PROJECT · PROJECT 37 OF 55

AI-BASED TRAFFIC VIOLATION DETECTION AND AUTOMATED LICENSE PLATE RECOGNITION SYSTEM

Domain: Intelligent Transportation & ANPR Surveillance | **Module:** project_37_traffic_violation

PROJECT ABSTRACT

As urban vehicle density continues to grow, enforcing traffic rules and maintaining road order have become increasingly challenging for law enforcement agencies. Traditional manual monitoring is insufficient for continuous surveillance and prone to human error. To address this, this study proposes an AI-based traffic violation detection and automated license plate recognition (ALPR) system. The system integrates advanced computer vision and deep learning algorithms to automatically monitor traffic cameras and detect common traffic infractions, such as red-light violations, speeding, illegal lane changes, and wrong-way driving. Object detection and tracking models, such as YOLO and DeepSORT, are utilized to identify vehicles, monitor their trajectories, and detect anomalous driving behaviors in real-time. Upon detecting a violation, the system automatically isolates the vehicle's region of interest and applies a dedicated deep learning-based ALPR model to extract the license plate characters under diverse environmental conditions, including poor lighting and motion blur. The system is validated on traffic video datasets, demonstrating high accuracy in both violation detection and license plate transcription. The recognized license plates are then matched against a database to generate automated citations or alert authorities. The deployment of this end-to-end automated solution improves enforcement efficiency, reduces the burden on manual surveillance, and acts as a strong deterrent against traffic violations, ultimately promoting safer driving behavior.

📌 Project Metadata & Specifications

Project Title	AI-BASED TRAFFIC VIOLATION DETECTION AND AUTOMATED LICENSE PLATE RECOGNITION SYSTEM
Domain / Specialization	Intelligent Transportation & ANPR Surveillance
Source Directory	`project_37_traffic_violation`
Backend REST API	`http://localhost:5037` (Flask Python)
Frontend Interface	Interactive Dashboard (`frontend/index.html`)
Execution Mode	Hybrid Dual-Mode (Online API & Offline Autonomous Simulation)

🏗️ System Methodology & Implementation Phases

PROJECT 37 — PHASE 1

Phase 1 — Requirements, Data and Prototype

The first phase will convert the project concept into a measurable prototype for computer vision/environmental or safety analytics. The work begins by defining the exact input, output, user flow and evaluation criteria from the project abstract. A small requirements specification will identify the primary prediction/detection task, expected classes or target value, acceptable response time, and the form in which results must be displayed.

1. Problem Definition and Scope

Define the operational scenario, target users, system boundaries, assumptions, success criteria and non-functional requirements. Separate what the model must predict from what the application must display or recommend. Identify cases that should be rejected, flagged for review, or returned with low confidence.

2. Data Preparation

Identify a suitable dataset or approved collection procedure matching the abstract. Build a data dictionary, remove unusable records, handle missing values, normalize or standardize numerical fields where appropriate, and verify labels. For image/video/audio/text projects, define preprocessing, augmentation and quality-control rules. Split data into training, validation and test sets without leakage.

3. Baseline Model and Experiment Setup

Implement a reproducible baseline using the algorithm/model family stated in the abstract when one is specified. Establish a simple reference result before introducing additional complexity. Record dataset version, preprocessing settings, model configuration, random seed, training time and evaluation metrics so that later improvements can be compared objectively.

4. Initial Application Prototype

Build the minimum user-facing workflow needed to demonstrate the core task. The prototype should accept valid input, run preprocessing, invoke the baseline model, and return the predicted result with an understandable confidence/score where supported. Keep model logic separate from the interface so the model can be replaced without rewriting the application.

Phase 1 Deliverables

Requirements and scope note; prepared dataset or data pipeline; baseline model; experiment log; initial application/prototype; and a short Phase 1 validation report documenting what works, what remains uncertain, and the measurements to be improved in Phase 2.

Project-Specific Phase 1 Work Items

- Create a labelled visual dataset structure and define image/video quality checks.
- Set a consistent image resize/normalization pipeline and establish augmentation rules.
- Record sample predictions visually so incorrect detections can be traced back to input regions.

PROJECT 37 — PHASE 2

Phase 2 — Optimization, Integration and Validation

The second phase will turn the Phase 1 prototype into a validated project for computer vision/environmental or safety analytics. The emphasis is on improving model reliability, connecting the complete workflow, testing realistic edge cases, and producing evidence that the final system meets the defined objectives.

1. Model Improvement

Compare relevant model configurations, tune important hyperparameters, address class imbalance where present, and evaluate whether additional features or augmentation improve generalization. Use validation

data for tuning and reserve the test set for final evaluation. Prefer measurable improvements over unnecessary model complexity.
2. End-to-End Integration
Connect the data/input layer, preprocessing, inference engine, result-processing logic, database or file storage where required, and user interface. Add input validation, error handling, logging and clear status messages. If recommendations, alerts, summaries, risk scores or dashboards are part of the abstract, implement them as a separate decision-support layer around the model.
3. Testing and Evaluation
Perform functional, model, integration and usability tests. Evaluate accuracy or regression error as appropriate, along with precision, recall, F1, confusion matrix, ROC-AUC, latency or other task-specific measures where relevant. Test difficult, noisy, incomplete and out-of-distribution inputs and document failure cases instead of hiding them.
4. Demonstration and Final Evidence
Prepare representative test cases, screenshots or result tables, model-comparison evidence and a final demonstration workflow. Document limitations, data constraints, privacy/security considerations where applicable, and future extensions. The final report should distinguish measured results from proposed future work.
Phase 2 Deliverables
Optimized model; integrated application; test and evaluation report; representative evidence; limitations and risk register; deployment/demo instructions; and final project documentation suitable for academic review or demonstration.
Project-Specific Phase 2 Validation
• Evaluate detection under changing lighting, weather, density, camera position or sensor conditions where relevant.
• Measure alert latency and false-alert behavior in addition to model accuracy.
• Demonstrate an operator-facing status/alert view with evidence supporting each detected event.
Final Expected Outcome
A reproducible, tested prototype that implements the project concept described in the supplied abstract, together with measurable evaluation evidence and clear documentation of limitations, assumptions and future enhancement opportunities.
can u find if any project in git hub can u get it and if u dont find a project u want to do the project and each project open new floder can u do it

The current local time is: 2026-10-06T20:33:06+05:30.

The user changed setting `Model Selection` from None to Claude Sonnet 4.6 (Thinking). No need to comment on this change if the user doesn't ask about it. If reporting what model you are, please use a human readable name instead of the exact string.

🚀 Local Deployment & Execution Guide

Follow the steps below to initialize and demonstrate the system:

1. Backend Server Setup

```
# Navigate to the backend directory cd "/Users/balamurugan/Documents/orbyte globel
/college project/project_37_traffic_violation/backend" # Install project dependencies pip
install -r requirements.txt # Run the Flask API service python3 app.py
```

The backend microservice will listen for requests at `http://localhost:5037`.

2. Frontend User Interface

- Open `/Users/balamurugan/Documents/orbyte globel /college project/project_37_traffic_violation/frontend/index.html` directly in Google Chrome, Firefox, or Safari.
- Alternatively, launch via the central **Master Launchpad Portal (`index.html`)** located at the root directory.
- The user interface is designed with **hybrid dual-mode resilience**: when the backend API is live, real-time inferences are queried; when offline, authentic client-side mathematical models simulate sensor telematics.